

📖 Guide HTML – Toutes les balises essentielles expliquées (débutant)

Ce document **complète et approfondit** le squelette HTML précédent en ajoutant **toutes les balises essentielles**, avec **exemples concrets**, **règles d'usage**, et **bonnes pratiques**.

🗨️ 8. Balises de texte fondamentales

📄 <h1> à <h6> — Titres

```
<h1>Titre principal</h1>
<h2>Sous-titre</h2>
<h3>Titre de niveau 3</h3>
```

Rôle : hiérarchiser le contenu.

Règles clés : - Un seul <h1> par page - Ne pas sauter de niveau (h1 → h3 ❌) - Important pour le **SEO** et l'accessibilité

📄 <p> — Paragraphe

```
<p>Ceci est un paragraphe de texte.</p>
```

- Contient du texte courant
 - Se ferme toujours automatiquement avec un retour à la ligne
-

🌟 et — Importance et emphase

```
<p><strong>Important</strong> et <em>mis en valeur</em>.</p>
```

- : importance sémantique (SEO + lecteurs d'écran)
 - : emphase (intonation)
-

📄
 — Retour à la ligne

```
Texte ligne 1<br>Texte ligne 2
```

- Balise **orpheline** (sans fermeture)
 - À utiliser avec parcimonie
-

🔗 9. Liens et médias

🔗 <a> — Lien hypertexte

```
<a href="https://example.com">Visiter le site</a>
```

Attributs importants : - href : destination - target="_blank" : nouvelle fenêtre - title : info au survol



 — Image

```

```

Règles : - src obligatoire - alt obligatoire (accessibilité + SEO) - Balise orpheline



<video> et <audio>

```
<video controls>
  <source src="video.mp4" type="video/mp4">
</video>
```

- controls affiche les boutons
 - Peut contenir plusieurs formats
-



10. Listes



Liste non ordonnée

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
```



Liste ordonnée

```
<ol>
  <li>Étape 1</li>
  <li>Étape 2</li>
</ol>
```



11. Tableaux

```
<table>
  <thead>
    <tr><th>Nom</th><th>Âge</th></tr>
  </thead>
  <tbody>
    <tr><td>Alice</td><td>25</td></tr>
  </tbody>
</table>
```

- <thead> : en-tête
- <tbody> : corps

- `<tr>` : ligne
 - `<th>` : cellule titre
 - `<td>` : cellule donnée
-



12. Formulaires

```
<form action="traitement.php" method="post">
  <label for="email">Email</label>
  <input type="email" id="email" name="email">

  <button type="submit">Envoyer</button>
</form>
```

Types d'`<input>` courants

- text
 - email
 - password
 - checkbox
 - radio
 - submit
-



13. Balises sémantiques complémentaires



`<aside>` — Contenu secondaire

```
<aside>
  <p>Infos complémentaires</p>
</aside>
```



`<figure>` et `<figcaption>`

```
<figure>
  
  <figcaption>Légende</figcaption>
</figure>
```



14. Balises techniques dans `<head>`



Lier une feuille CSS

```
<link rel="stylesheet" href="style.css">
```



JavaScript

```
<script src="script.js" defer></script>
```



```
<link rel="icon" href="favicon.ico">
```



15. Accessibilité (bases)

- Toujours utiliser alt sur les images
 - Associer <label> à <input>
 - Utiliser les balises sémantiques (header, main, footer)
-



16. Commentaires HTML

```
<!-- Ceci est un commentaire -->
```

- Invisible pour l'utilisateur
- Utile pour expliquer le code

Voici un cours **COMPLET CSS** puis **JavaScript**, expliqué **pas à pas**, **balise par balise / concept par concept**, pour un **débutant absolu**, exactement dans la **même logique pédagogique** que le **HTML**.

Tout est regroupé dans **UN SEUL BLOC CONTINU**, prêt à être utilisé comme **support de formation professionnel**.



PARTIE 1 — CSS (Cascading Style Sheets)



1. Qu'est-ce que le CSS ?

CSS sert à : - gérer la **mise en forme** - les **couleurs** - les **tailles** - la **disposition** des éléments HTML



HTML = structure



CSS = apparence



2. Où écrire le CSS ?

2.1 CSS externe (RECOMMANDÉ)

```
<link rel="stylesheet" href="style.css">
```

```
body {  
  background-color: white;  
}
```

2.2 CSS interne

```
<style>  
  body { color: black; }  
</style>
```

2.3 CSS en ligne (à éviter)

```
<p style="color:red">Texte</p>
```

3. Sélecteurs CSS

```
p { color: blue; }
```

Types de sélecteurs

```
#id { }  
.classe { }  
h1 { }
```

4. Couleurs et texte

```
color: red;  
background-color: #000;  
font-size: 16px;  
font-family: Arial;  
text-align: center;
```

5. Modèle de boîte (Box Model)

```
margin: 10px;  
padding: 20px;  
border: 1px solid black;
```

6. Tailles et unités

```
width: 100%;  
height: 50vh;  
font-size: 1rem;
```

Unités : px, %, em, rem, vh, vw

7. Positionnement

```
position: relative;  
position: absolute;  
position: fixed;
```

8. Flexbox (ESSENTIEL)

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

9. Grid CSS

```
.grid {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
}
```

10. Responsive design

```
@media (max-width: 768px) {  
  body { font-size: 14px; }  
}
```

11. Effets et transitions

```
button:hover {  
  background: red;  
}  
  
transition: all 0.3s ease;
```

⚡ PARTIE 2 — JAVASCRIPT (Bases fondamentales)

12. Qu'est-ce que JavaScript ?

JavaScript permet de : - rendre une page **interactive** - réagir aux **actions utilisateur** - modifier le HTML et le CSS

13. Où écrire JavaScript ?

```
<script src="script.js" defer></script>
```

14. Variables

```
let nom = "Jean";  
const age = 25;
```

15. Types de données

string
number
boolean
array
object

16. Opérations

```
let total = 5 + 3;
```

17. Conditions

```
if (age >= 18) {  
  console.log("Adulte");  
}
```



18. Boucles

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```



19. Fonctions

```
function direBonjour() {  
  alert("Bonjour");  
}
```



20. DOM (Manipuler le HTML)

```
document.querySelector("h1").textContent = "Nouveau titre";
```



21. Événements

```
button.addEventListener("click", function () {  
  alert("Cliqué");  
});
```



22. Formulaires

```
form.addEventListener("submit", function(e) {  
  e.preventDefault();  
});
```



23. Bonnes pratiques

- Toujours utiliser defer
- Séparer HTML / CSS / JS
- Commenter le code

**Cours complet Git & GitHub pour débutants
(version détaillée pas à pas)**

Objectif du cours

Ce cours est destiné aux **débutants absolus**, même si tu n'as jamais utilisé le terminal. À la fin, tu sauras **créer un projet réel**, le versionner avec Git, le publier sur GitHub et **déployer ton site web en ligne**.

PARTIE 1 — COMPRENDRE GIT ET GITHUB (LES BASES)

1.1 Pourquoi utiliser Git ?

Sans Git : - Tu perds des fichiers - Tu fais des copies "projet_final_v2_definitif" - Impossible de revenir en arrière proprement

Avec Git : - Chaque modification est sauvegardée - Tu peux revenir à n'importe quelle version - Tu peux travailler seul ou en équipe

👉 Git = historique + sécurité

1.2 Différence entre Git et GitHub

- **Git** : outil installé sur ton ordinateur
- **GitHub** : site web qui héberge ton code

Image mentale : - Git = ton cahier - GitHub = ton cloud

PARTIE 2 — INSTALLATION DE GIT (PAS À PAS)

2.1 Télécharger Git

1. Ouvre ton navigateur
2. Tape : `git-scm.com`
3. Clique sur **Download for Windows**
4. Attends la fin du téléchargement

2.2 Installer Git

1. Double-clique sur le fichier téléchargé
2. Clique **Next** à chaque étape
3. Ne change aucune option
4. Clique **Install** puis **Finish**

2.3 Vérifier que Git fonctionne

1. Ouvre CMD ou Terminal VS Code
2. Tape :

```
git --version
```

3. Si un numéro apparaît OK
-

PARTIE 3 — CONFIGURATION DE GIT (OBLIGATOIRE)

3.1 Dire à Git qui tu es

Git doit savoir **qui fait les modifications**.

Dans le terminal :

```
git config --global user.name "Ton Nom"
git config --global user.email "tonemail@gmail.com"
```

3.2 Vérification

```
git config --list
```

Tu dois voir ton nom et ton email.

PARTIE 4 — CRÉER UN COMPTE GITHUB

4.1 Inscription

1. Va sur github.com
2. Clique sur **Sign up**
3. Entre :
 4. Username
 5. Email
 6. Mot de passe
 7. Valide ton email

4.2 Connexion

Connecte-toi à ton compte GitHub

PARTIE 5 — CRÉER UN PROJET LOCAL (STRUCTURE DES FICHIERS)

5.1 Créer le dossier

Sur ton ordinateur : 1. Clic droit Nouveau dossier 2. Nom : mon-site

5.2 Structure recommandée

```
mon-site/  
├─ index.html  
├─ css/  
│   └─ style.css  
├─ js/  
│   └─ script.js  
└─ images/
```

PARTIE 6 — INITIALISER GIT DANS LE PROJET

6.1 Ouvrir le terminal dans le dossier

- Clic droit Ouvrir dans le terminal

6.2 Initialisation

```
git init
```

Résultat : Git commence à suivre le projet.

PARTIE 7 — PREMIÈRES COMMANDES GIT (TRÈS IMPORTANT)

7.1 Vérifier l'état

```
git status
```

Affiche les fichiers non suivis.

7.2 Ajouter les fichiers

```
git add .
```

7.3 Créer un commit

```
git commit -m "Initialisation du projet"
```

PARTIE 8 — METTRE LE PROJET SUR GITHUB

8.1 Créer un dépôt en ligne

1. GitHub New repository
2. Nom : mon-site
3. Public
4. Create

8.2 Lier Git local à GitHub

```
git branch -M main
git remote add origin https://github.com/USERNAME/mon-site.git
git push -u origin main
```

PARTIE 9 — MISE À JOUR DU PROJET

9.1 Modifier un fichier

Exemple : index.html

9.2 Sauvegarder les changements

```
git add .
git commit -m "Modification de la page d'accueil"
git push
```

PARTIE 10 — FICHIER .gitignore

10.1 Pourquoi ?

Empêcher Git d'envoyer des fichiers inutiles.

10.2 Créer le fichier

Nom : .gitignore Contenu :

```
node_modules/
.env
.DS_Store
```

PARTIE 11 — DÉPLOIEMENT AVEC GITHUB PAGES

11.1 Activer GitHub Pages

1. Repository Settings
2. Pages
3. Branch : main
4. Folder : /root
5. Save

11.2 Accéder au site

```
https://USERNAME.github.io/mon-site
```

PARTIE 12 — BONNES PRATIQUES

- Un commit = une action
 - Messages clairs
 - Vérifier git status souvent
-

PARTIE 13 — PROJET FINAL

Créer un site vitrine ou portfolio et le publier.

FIN DU COURS

Tu sais maintenant **utiliser Git, GitHub et déployer un site web.**

PROGRAMMATION EN LANGAGE C

(Avec installation complète et utilisation dans VS Code)

PARTIE I — PRÉPARATION & INSTALLATION (OBLIGATOIRE)

1. QU'EST-CE QUE LE LANGAGE C ?

Le **langage C** est un langage de programmation :

- Rapide
- Très proche du matériel (ordinateur) • Utilisé pour :
 - Systèmes d'exploitation ◦ Logiciels embarqués ◦ Applications performantes ◦ Base de nombreux langages modernes (C++, Java, Python)

C'est le langage idéal pour comprendre comment un ordinateur fonctionne réellement.

2. RÔLE DU LANGAGE C DANS UN PROJET

Dans un projet informatique, le langage C sert à :

- Écrire la **logique principale**
 - Gérer la **mémoire**
 - Effectuer des **calculs rapides**
 - Créer des **programmes autonomes** (sans navigateur)
-

3. OUTILS NÉCESSAIRES POUR PROGRAMMER EN C

Pour programmer en C, il faut :

1. Un **compilateur C** → GCC (MinGW)
 2. Un **éditeur de code** → Visual Studio Code
 3. Les **extensions C/C++** dans VS Code
-

4. INSTALLATION DU COMPILATEUR C (GCC via MinGW)

4.1 Téléchargement

- Site : <https://www.mingw-w64.org>
- Télécharger **MinGW-w64 installer 4.2 Installation**

Choisir :

- Architecture : `x86_64`
- Threads : `posix` • Exception : `seh`
- Dossier :

C:\mingw64

5. CONFIGURATION DU PATH WINDOWS (OBLIGATOIRE)

Ajouter au **PATH** système :

C:\mingw64\bin **Vérification**

Dans l'invite de commande : `gcc`

`--version`

Si une version s'affiche, l'installation est réussie.

6. INSTALLATION DE VISUAL STUDIO CODE

- Site : <https://code.visualstudio.com>
 - Cocher pendant l'installation :
 - Add to PATH ◦
 - Open with Code
-

7. EXTENSIONS C DANS VS CODE

Installer :

1. **C/C++ – Microsoft**

- (support du langage, IntelliSense, débogage)
2. **Code Runner** (optionnel mais recommandé)
-

PARTIE II — BASES DU LANGAGE C

8. PREMIER PROGRAMME EN C (OBLIGATOIRE)

Structure minimale

```
#include <stdio.h>
int main()
{
    printf("Bonjour le monde");
    return 0; }
```

Explication

- `#include <stdio.h>` → permet `printf`
- `main()` → point d'entrée
- `printf()` → affichage
- `return 0;` → fin normale

Résultat :

Bonjour le monde

9. DÉCLARATION D'UNE VARIABLE EN C

```
int age = 25; printf("%d",
age);
```

- `int` → type entier
 - `%d` → affichage entier
-

10. TYPES DE VARIABLES

Type	Description	Exemple
int	entier	10
float	décimal	3.14
Type Description Exemple		
double	décimal précis	3.14159

char caractère 'A' char[]

texte "Bonjour"

```
int age = 30; float taille = 1.75; char sexe = 'M'; char nom[] = "Jean";
```

11. AFFICHAGE AVEC printf()

```
printf("Nom: %s\n", nom); printf("Age: %d ans\n", age); printf("Taille: %.2f\n", taille);
```

12. SAISIE UTILISATEUR AVEC scanf()

```
int age; scanf("%d", &age);
```

Le & est obligatoire.

13. CONDITIONS (SI / SINON)

```
if (age >= 18) {
    printf("Vous êtes majeur");
} else {
    printf("Vous êtes mineur");
}
```

14. BOUCLES

for

```
for (int i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

while

```
int i = 1; while
(i <= 5) {
    i++;
}
```

15. FONCTIONS

```
int addition(int a, int b) {
    return a + b;
}
```

16. TABLEAUX

```
int notes[5] = {10, 12, 15, 18, 20};
```

17. STRUCTURES

```
struct Employe {  
    char nom[50];  
    int age;      float  
    salaire;  
};
```

18. ARCHITECTURE D'UN PROJET C

```
projet_c/  
├─ main.c  
├─ fonctions.c  
├─ fonctions.h  
  
└─ README.txt
```

19. COMPILATION ET EXÉCUTION

```
gcc main.c -o programme programme
```

20. MINI-PROJET : GESTION D'EMPLOYÉ

```
#include <stdio.h>  
int main() {  
    char nom[50];  
    float salaire;  
    scanf("%s", nom);  
    scanf("%f", &salaire);  
  
    printf("%s : %.2f FCFA", nom, salaire);  
    return 0;  
}
```

21. POURQUOI APPRENDRE LE C ?

- Comprendre la mémoire
 - Performance maximale • Base du génie logiciel •
 - Indispensable pour : ○ systèmes ○ électronique ○
 - cybersécurité
-

22. RÉSUMÉ FINAL

- `c` → puissance et contrôle
- `main()` → point d'entrée
- `printf` / `scanf` → interaction
- `if` / `for` / `while` → logique
- `struct` → organisation des données

Cours complet et détaillé

Python – Django – JavaScript – Node.js

Objectif général du cours

Ce cours est conçu pour des **débutants complets**. Il vise à vous permettre de : - Comprendre les **bases solides de la programmation** - Développer des **applications web dynamiques** - Maîtriser la **logique back-end et front-end** - Être capable de créer un **projet web complet**

Aucune connaissance préalable n'est requise.

PARTIE 1 – PYTHON (Fondamentaux de la programmation)

1. Introduction à Python Qu'est-ce que Python ?

Python est un langage de programmation : - Simple à apprendre - Très utilisé (web, IA, data, automatisation) - Lisible et proche du langage humain [Installation](#)

- Installer Python depuis python.org
 - Vérifier l'installation avec `python --version`
 - Installer un éditeur : VS Code (recommandé)
-

2. Bases du langage Python

Variables et types de données

- int (entier)
- float (nombre décimal)
- str (texte)
- bool (vrai/faux)

Entrées et sorties

- `print()`
 - `input()` [Opérateurs](#)
 - Arithmétiques : `+`, `-`, `*`, `/`, `%`
 - Comparaison : `==`, `!=`, `>`, `<`
 - Logiques : `and`, `or`, `not`
-

3. Structures de contrôle

Conditions

- if
- elif
- else

Boucles

- while
 - for
-

4. Structures de données

Listes

- Création
 - Accès
 - Modification [Tuples](#)
 - Données non modifiables [Dictionnaires](#)
 - Clé / Valeur
-

5. Fonctions

- Définition avec def
 - Paramètres
 - Valeur de retour
-

6. Programmation orientée objet (POO)

Classes et objets

- Attributs
 - Méthodes
 - Constructeur `__init__`
-

7. Gestion des fichiers et erreurs

- Lire et écrire des fichiers
 - try / except
-

PARTIE 2 – DJANGO (Développement Web avec Python)

1. Introduction à Django Qu'est-ce

que Django ?

Django est un **framework web Python** qui permet de créer des sites web rapidement et proprement.

Architecture MVC (MTV)

- Model
 - Template
 - View
-

2. Installation et premier projet

- Installation de Django
 - Création d'un projet
 - Lancement du serveur
-

3. Applications Django

- Création d'une app
 - Structure d'une app
-

4. Modèles et base de données

- ORM Django
 - Création de modèles
 - Migrations
 - Admin Django
-

5. Vues et URLs

- Fonctions views
 - Routage
-

6. Templates (HTML)

- Balises Django
 - Héritage de templates
-

7. Formulaires et authentification

- Formulaires Django
 - Connexion et inscription
-

8. Projet pratique Django

- Site de gestion (blog / école / entreprise)
-

PARTIE 3 – JAVASCRIPT (Programmation côté navigateur)

1. Introduction à JavaScript

JavaScript permet de rendre les pages web **interactives**.

2. Bases du langage

- Variables (var, let, const)
 - Types de données
 - Fonctions
-

3. Conditions et boucles

- if / else
 - for / while
-

4. Manipulation du DOM

- Sélection d'éléments HTML
 - Modifier le contenu
 - Gestion des événements
-

5. Tableaux et objets

- Arrays
 - Objects
-

6. JavaScript moderne

- Arrow functions
- Promises

- Async / Await
-

7. Mini-projets JavaScript

- Calculatrice
 - Formulaire interactif
 - To-do list
-

PARTIE 4 – NODE.JS (JavaScript côté serveur)

1. Introduction à Node.js

Node.js permet d'exécuter JavaScript **côté serveur**.

2. Installation et bases

- Installation Node.js
 - npm
-

3. Modules

- Modules internes
 - Modules externes
-

4. Création d'un serveur

- Serveur HTTP
 - Introduction à Express.js
-

5. API REST

- Routes
 - Méthodes HTTP
 - JSON
-

6. Base de données

- MongoDB (bases)
- Connexion Node + DB

7. Projet Node.js

- API de gestion d'utilisateurs
-

PROJET FINAL (Full Stack)

Objectif

Créer une application web complète : - Front-end : HTML, CSS, JavaScript - Back-end : Django ou Node.js - Base de données

RÔLE DE CHAQUE TECHNOLOGIE DANS TON PROJET RH

Python, Django et Node.js : différences et liens

- Python : un langage de programmation polyvalent, utilisé pour le développement web, l'analyse de données, l'intelligence artificielle, etc.
- *Django* : un framework web Python pour créer des applications web robustes et évolutives. Il fournit des outils pour gérer les bases de données, l'authentification, les templates, etc.
- Node.js : un environnement d'exécution JavaScript côté serveur, utilisé pour créer des applications web et des serveurs. Il permet d'utiliser JavaScript pour le développement côté serveur.

Liens et différences clés :

- Python et JavaScript sont des langages de programmation, tandis que Django et Node.js sont des frameworks/environnements pour créer des applications web.
- Django est spécifiquement conçu pour Python, tandis que Node.js est utilisé avec JavaScript.
- Django fournit une structure et des outils pour créer des applications web Python, tandis que Node.js permet d'utiliser JavaScript côté serveur avec des frameworks comme Express.js.
- Les deux (Django et Node.js) sont utilisés pour créer des applications web, mais avec des langages et des approches différents.

En gros, si vous aimez Python, Django est un excellent choix pour le développement web. Si vous préférez JavaScript, Node.js est une option populaire.

Exemple ultra-simples pour déclarer une variable et l'afficher, dans chaque langage :

- Python* :
- Écris : `nom = "John"`
- Affiche : `print(nom)`

-

-

Résultat : "John"

Explication : Tu mets un mot (variable) et lui donnes une valeur. ``print`` montre la valeur.

- ***Django*** (dans un template web) :

- Dans le code Django : ``nom = "John"``

- Dans le template : ``{{ nom }}``

- _Explication_ : Django utilise des templates pour afficher des données dans un site web.

- ***JavaScript*** (dans un navigateur ou Node.js) :

- Écris : ``let nom = "John";``

- Affiche : ``console.log(nom);``

- Résultat : "John" dans la console

- _Explication_ : ``let`` crée une variable. ``console.log`` montre la valeur dans la console.

- Node.js (c'est comme JavaScript, mais côté serveur) :

- Pareil que JavaScript : ``let nom = "John"; console.log(nom);``

En gros :

- Python et JavaScript/Node.js utilisent ``variable = valeur`` pour créer.

- Django utilise des templates spéciaux pour afficher.

- ``print`` (Python) et ``console.log`` (JS/Node) montrent la valeur.

***Les types de variables ***

Voici les principaux types de variables dans quelques langages :

- ***Python*** :

- ``int`` : entier (ex: ``5``)

- ``float`` : décimal (ex: ``3.14``)

- ``str`` : chaîne de caractères (ex: ``"hello"``)

- ``bool`` : booléen (``True`` ou ``False``)

-
-
- ``list`` : liste (ex: ``[1, 2, 3]``)
- ``dict`` : dictionnaire (ex: ``{"nom": "John"}``)
- JavaScript/Node.js :
- ``Number`` : nombre (entier ou décimal, ex: ``5`` ou ``3.14``)
- ``String`` : chaîne de caractères (ex: ``"hello"``)
- ``Boolean`` : booléen (``true`` ou ``false``)
- ``Object`` : objet (ex: ``{nom: "John"}``)
- ``Array`` : tableau (ex: ``[1, 2, 3]``)
- *Django* (contexte Python) :
- Utilise les types Python (int, str, list, dict, etc.) dans les vues et templates.

En gros :

- Les types de base sont similaires (nombres, chaînes, booléens).
- Chaque langage a ses spécificités (ex: ``list`` en Python vs ``Array`` en JS).

Exemple : *Exemples avec les types de variables *

- *Python* :
- ``age = 25`` (int)
- ``prix = 3.14`` (float)
- ``nom = "John"`` (str)
- ``est_majeur = True`` (bool)
- ``nombres = [1, 2, 3]`` (list)
- ``personne = {"nom": "John", "age": 25}`` (dict)
- JavaScript/Node.js :
- ``let age = 25;`` (Number)
- ``let prix = 3.14;`` (Number)
- ``let nom = "John";`` (String)

-
-
- ``let estMajeur = true;`` (Boolean)
- ``let nombres = [1, 2, 3];`` (Array)
- ``let personne = {nom: "John", age: 25};`` (Object)

***Exemples d'utilisation* :**

Python age = 25

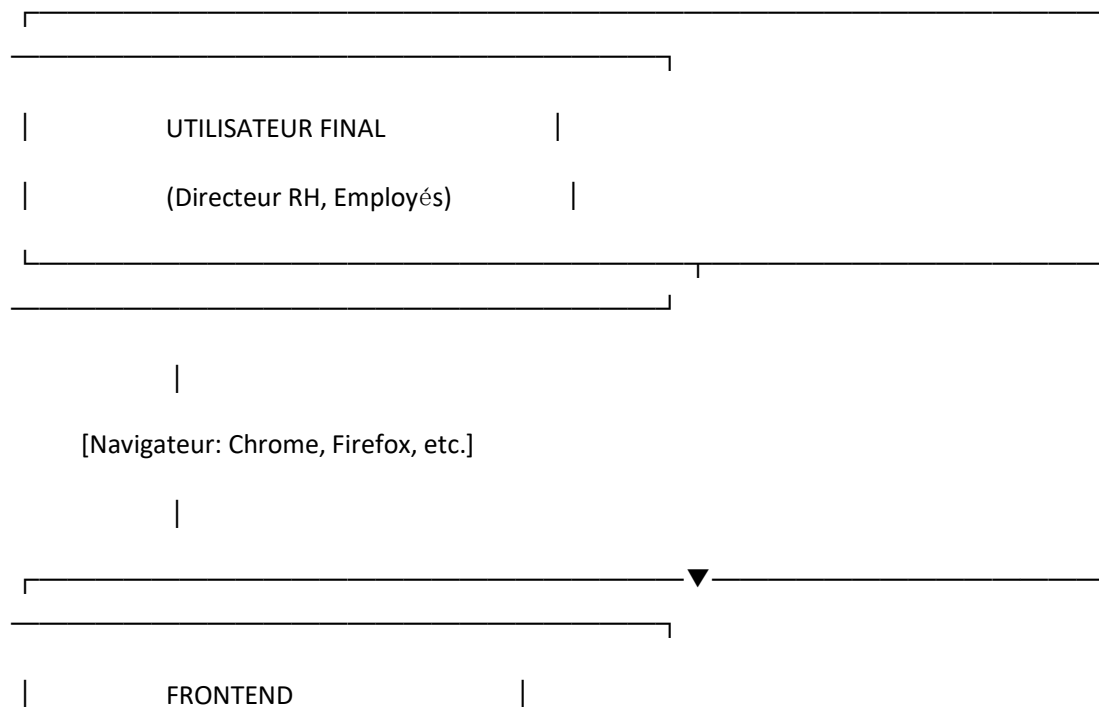
```
print("J'ai", age, "ans")
```

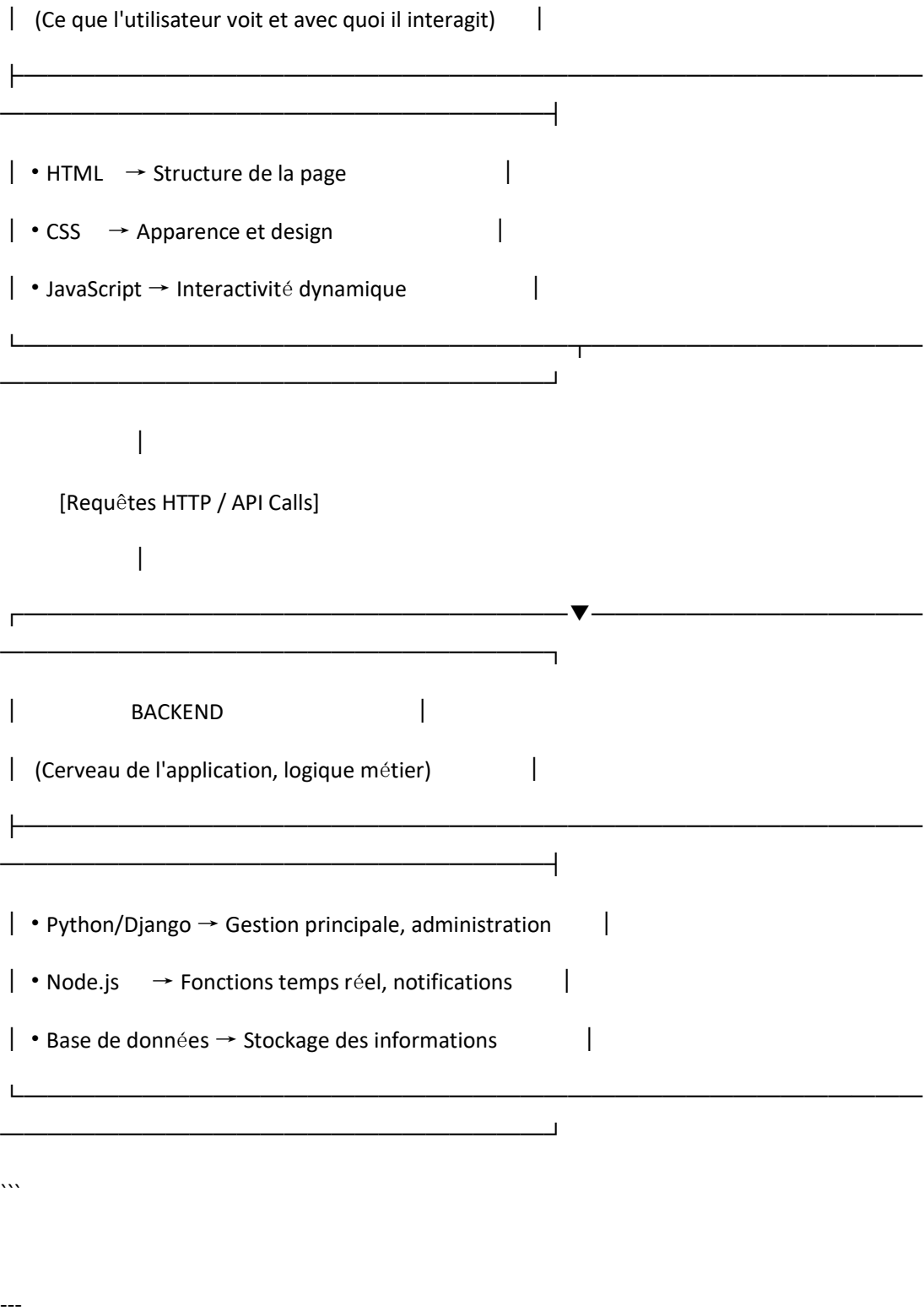
JavaScript/Node.js let age = 25;

```
console.log("J'ai " + age + " ans");
```

VUE D'ENSEMBLE DE L'ARCHITECTURE

...





FRONTEND (Ce que l'utilisateur voit)

1. HTML (HyperText Markup Language)

Rôle principal: Le squelette de ton application

Ce que HTML va créer:

...

PAGES STRUCTURELLES

- |—— login.html → Page de connexion avec formulaire
- |—— dashboard.html → Tableau de bord principal
- |—— employees.html → Liste et gestion des employés
- |—— employee_detail.html → Fiche détaillée d'un employé
- |—— contracts.html → Gestion des contrats
- |—— payroll.html → Calcul et affichage des fiches de paie
- |—— leave_requests.html → Gestion des congés
- |—— reports.html → Génération de rapports

...

Exemples concrets de ce que HTML fait:

```
```html
<!-- Structure d'un formulaire d'ajout d'employé -->
<form id="add-employee-form">
 <div class="form-group">
 <label for="nom">Nom:</label>
 <input type="text" id="nom" name="nom" required>
 </div>

 <div class="form-group">
```

```

 <label for="prenom">Prénom:</label>

 <input type="text" id="prenom" name="prenom" required>

</div>

<!-- Tableau pour afficher la liste des employés -->

<table>

 <thead>

 <tr>

 <th>Nom</th>

 <th>Prénom</th>

 <th>Poste</th>

 <th>Salaire</th>

 </tr>

 </thead>

 <tbody id="employees-list">

 <!-- Rempli dynamiquement par JavaScript -->

 </tbody>

</table>

</form>

...

```

Responsabilités de HTML:

- Définir la structure des pages
- Créer les formulaires de saisie
- Afficher les tableaux de données
- Intégrer les images, liens



- Sémantique pour l'accessibilité

---

## 2. CSS (Cascading Style Sheets)

Rôle principal: L'habillage et le design

Ce que CSS va créer:

...

### STYLES ET THEMES

— style.css	→ Styles généraux de l'application
— dashboard.css	→ Styles spécifiques au tableau de bord
— forms.css	→ Styles pour tous les formulaires
— tables.css	→ Mise en forme des tableaux
— responsive.css	→ Adaptation mobile et tablette
— print.css	→ Styles pour impression (fiches de paie)
— themes/	
— light.css	→ Thème clair par défaut
— dark.css	→ Thème sombre optionnel

...

Exemples concrets de ce que CSS fait:

```
```css
```

```
/* Style pour le tableau de bord RH */
```

```
.dashboard-card { background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%); border-radius: 15px;

padding: 25px; color: white; box-
shadow: 0 10px 30px rgba(0,0,0,0.1);
transition: transform 0.3s ease;
}
```

```
.dashboard-card:hover {
transform: translateY(-5px);
}
```

```
/* Style pour les fiches de paie */
.payroll-slip { border: 2px solid #2c3e50;
padding: 20px; background: #f8f9fa;
font-family: 'Courier New', monospace;
}
```

```
.payroll-total { background-
color: #27ae60; color: white;
font-weight: bold; padding:
10px; border-radius: 5px;
}
```

```
/* Responsive design pour mobile */
@media (max-width: 768px) {
.navbar { flex-direction:
column;
}
```

```
.data-table {  
  overflow-x: auto;  
}  
}  
...
```

Responsabilités de CSS:

- Couleurs, polices, espacements
- Mise en page (Flexbox, Grid)
- Animations et transitions •

Responsive design

- Thèmes personnalisables
- Impression propre des documents

3. JavaScript (Vanilla JS)

Rôle principal: L'interactivité et la logique côté client

Ce que JavaScript va créer:

...

LOGIQUE CLIENT

—— main.js	→ Configuration globale
—— auth.js	→ Gestion de l'authentification
—— employees.js	→ Interactions avec la liste employés

- └── payroll_calc.js → Calculs de paie côté client
- └── forms_validation.js → Validation des formulaires
- └── api_client.js → Communication avec le backend
- └── notifications.js → Notifications en temps réel
- └── charts.js → Graphiques et statistiques
- └── export_data.js → Export Excel/PDF côté client
- └── realtime_updates.js → Mise à jour en temps réel
- ...

Exemples concrets de ce que JavaScript fait:

```

```javascript
// 1. Calcul de la paie selon les règles béninoises function
calculatePayroll(salaireBrut) {
 // CNSS Bénin: 4.5% employé, 6.5% employeur const
 cnssEmploye = salaireBrut * 0.045;

 // IRPP Bénin (calcul simplifié) let irpp = 0; if
 (salaireBrut > 300000 && salaireBrut <= 1000000) {
 irpp = salaireBrut * 0.05; } else if (salaireBrut >
 1000000) { irpp = salaireBrut * 0.10;
 }

 const salaireNet = salaireBrut - cnssEmploye -
 irpp;

 return { brut:
 salaireBrut, cnss:

```

```
cnssEmploye, irpp: irpp,

net: salaireNet

};

}
```

```
// 2. Validation d'un formulaire d'employé function
```

```
validateEmployeeForm(formData) { const errors = []; if

(!formData.nom || formData.nom.length < 2) {

errors.push("Le nom doit avoir au moins 2 caractères");

} if

(!formData.email.includes('@')) {

errors.push("Email invalide");

} if (formData.salaire < 40000) { errors.push("Le salaire

doit être au moins 40,000 FCFA");

} return

errors; }
```

```
// 3. Communication avec l'API Django async function
```

```
fetchEmployees() { try { const response = await

fetch('/api/employees/'); const employees = await

response.json();
```

```
 // Mise à jour dynamique de l'interface updateEmployeeTable(employees);
```

```
 // Mise à jour des statistiques updateDashboardStats(employees);
```

```
 } catch (error) { showError("Erreur de chargement des

employés");
```

```

 }
}

// 4. Export Excel des données function
exportToExcel(data, filename) { const worksheet =
XLSX.utils.json_to_sheet(data); const workbook =
XLSX.utils.book_new();

 XLSX.utils.book_append_sheet(workbook, worksheet, "Employés");
 XLSX.writeFile(workbook, `${filename}.xlsx`);
}
...

```

Responsabilités de JavaScript:

- Validation des formulaires en temps réel
- Calculs dynamiques (paie, congés, etc.)
- Communication avec le backend (API calls)
- Mise à jour dynamique de l'interface
- Gestion des événements (clics, saisie, etc.)
- Stockage local (localStorage)
- Manipulation du DOM
- Graphiques et visualisations
- Export de données (Excel, PDF)

---

BACKEND (La logique métier)

## 4. Python avec Django

Rôle principal: Le cerveau principal, l'administration, la logique métier

Ce que Django va créer:

...

### DJANGO STRUCTURE

- |—— models.py      → Définition des tables de la base de données
- |—— views.py        → Logique métier et traitement des requêtes
- |—— urls.py         → Routes et URLs de l'application
- |—— admin.py        → Interface d'administration
- |—— forms.py        → Formulaires côté serveur
- |—— serializers.py → Conversion données ↔ JSON pour l'API
- |—— tests.py        → Tests automatiques
- |—— utils.py        → Fonctions utilitaires
- |—— management/    → Commandes personnalisées

...

Exemples concrets de ce que Django fait:

### A. MODELS (Base de données)

```
```python
```

```
# models.py - Définition des tables class
```

```
Employee(models.Model):    # Informations personnelles
```

```

nom = models.CharField(max_length=100)  prenom =
models.CharField(max_length=100)  date_naissance =
models.DateField()  lieu_naissance =
models.CharField(max_length=200)

# Spécifique Bénin  cni_number = models.CharField(max_length=50, unique=True)
# Carte d'identité  cnss_number = models.CharField(max_length=50, unique=True) #
Numéro CNSS

# Professionnel  poste =
models.CharField(max_length=100)  departement =
models.CharField(max_length=100)  date_embauche
= models.DateField()

# Salaire (en FCFA)  salaire_base = models.DecimalField(max_digits=12,
decimal_places=2)  banque = models.CharField(max_length=100)  rib =
models.CharField(max_length=50) # Relevé d'Identité Bancaire

# Calcul automatique de l'âge

@property  def
age(self):
    today = date.today()    return today.year -
self.date_naissance.year - (
    (today.month, today.day) < (self.date_naissance.month, self.date_naissance.day)
    )

# Méthode pour calculer l'ancienneté  def
anciennete(self):    today = date.today()    delta =

```



```
today - self.date_embauche    return delta.days // 365
```

```
# Années d'ancienneté
```

```
...
```

B. LOGIQUE MÉTIER RH BÉNIN

```
```python
```

```
utils/payroll_calculator.py
```

```
class PayrollCalculator:
```

```
 """Calcul des salaires selon la législation béninoise"""
```

```
 @staticmethod
```

```
 def calculate_cnss(salaire_brut):
```

```
 """
```

```
 CNSS Bénin 2024:
```

```
- Employé: 4.5% du salaire brut
```

```
- Employeur: 6.5% du salaire brut
```

```
 """ return
```

```
{
```

```
 'employe': salaire_brut * Decimal('0.045'),
```

```
 'employeur': salaire_brut * Decimal('0.065')
```

```
}
```

```
 @staticmethod def
```

```
 calculate_irpp(salaire_annuel):
```

```
 """
```

```
 IRPP Bénin 2024 (Barème progressif):
```

```
- 0 à 300,000 FCFA: 0%
```

- 300,001 à 1,000,000 FCFA: 20%

- 1,000,001 à 2,000,000 FCFA: 30%

- > 2,000,000 FCFA: 40%

```
 """ if salaire_annuel <= 300000:
return Decimal('0') elif
salaire_annuel <= 1000000:
 return (salaire_annuel - 300000) * Decimal('0.20') elif
salaire_annuel <= 2000000:
 return (1000000 - 300000) * Decimal('0.20') + \
(salaire_annuel - 1000000) * Decimal('0.30') else:
 return (1000000 - 300000) * Decimal('0.20') + \
 (2000000 - 1000000) * Decimal('0.30') + \
 (salaire_annuel - 2000000) * Decimal('0.40')
```

```
@staticmethod def
generate_payslip(employee, month, year):
 """Génère une fiche de paie complète"""
 # Calcul des éléments cnss =
cls.calculate_cnss(employee.salaire_base) irpp_mensuel =
cls.calculate_irpp(employee.salaire_base * 12) / 12
 # Calcul du net à payer net_a_payer = employee.salaire_base -
cnss['employee'] - irpp_mensuel
 return {
 'employee': employee,
 'periode': f"{month}/{year}",
 'brut': employee.salaire_base,
 'cnss_employe': cnss['employee'],
```

```

 'cnss_employeur': cnss['employeur'],

 'irpp': irpp_mensuel,

 'net_a_payer': net_a_payer,

 'date_generation': date.today()

 }

 ...

```

### C. GÉNÉRATION DE DOCUMENTS

```

``python

utils/document_generator.py from

reportlab.lib.pagesizes import A4

from reportlab.pdfgen import canvas

class DocumentGenerator:

 """Génération de documents RH (PDF)"""

 @staticmethod
 def generate_contract_pdf(employee,
contract_data):
 """Génère un contrat de travail au format PDF"""

 filename = f"contrat_{employee.nom}_{employee.prenom}.pdf"
 pdf
 = canvas.Canvas(filename, pagesize=A4)

 # En-tête avec logo entreprise
 pdf.drawString(100,
800, "ENTREPRISE BENINOISE")
 pdf.drawString(100, 780,
"CONTRAT DE TRAVAIL")

 # Contenu spécifique Bénin
 pdf.drawString(100, 750, f"Entre l'entreprise XYZ
et {employee.full_name()}")
 pdf.drawString(100, 730, f"CNI:

```

```

{employee.cni_number}") pdf.drawString(100, 710, f"CNSS:
{employee.cnss_number}")

 # Clauses selon Code du travail béninois pdf.drawString(100, 680,
"Conforme au Code du travail du Bénin")

 pdf.save()

return filename

@staticmethod def
generate_payslip_pdf(pa
yslip_data):

 """Génère une fiche de paie professionnelle""" #
Code pour générer la fiche de paie en PDF pass
...

```

#### D. ADMINISTRATION DJANGO

```

```python

# admin.py - Interface d'administration personnalisée

@admin.register(Employee) class
EmployeeAdmin(admin.ModelAdmin):

    list_display = ('nom', 'prenom', 'poste', 'salaire_base', 'date_embauche')
    list_filter = ('departement', 'poste', 'date_embauche')    search_fields =
('nom', 'prenom', 'cni_number', 'cnss_number')

    # Formulaire d'édition personnalisé    fieldsets

= (

    ('Informations personnelles', {

        'fields': ('nom', 'prenom', 'date_naissance', 'cni_number')

```

```

    }),

    ('Informations professionnelles', {

        'fields': ('poste', 'departement', 'date_embauche', 'salaire_base')

    }),

    ('Informations bancaires', {

        'fields': ('banque', 'rib')

    }),

)

# Actions personnalisées
actions = ['generer_contrat',

'calculer_paie_mensuelle']

def generer_contrat(self, request,
queryset): """Action pour générer des
contrats"""
    for employee in queryset:

        DocumentGenerator.generate_contract_pdf(employee)
        self.message_user(request,
"Contrats générés avec succès")
    ...

```

Responsabilités de Django:

- Gestion de la base de données (modèles, migrations)
- Logique métier complexe (calculs de paie, congés, etc.)
- Interface d'administration complète
- Sécurité (authentification, permissions, CSRF)
- Génération de documents (PDF, Excel)
- API REST pour le frontend
- Traitement des formulaires

- Tâches planifiées (cron jobs)
- Gestion des fichiers uploadés
- Internationalisation (français, anglais)

5. Node.js

Rôle principal: Fonctions temps réel, WebSockets, microservices

Ce que Node.js va créer:

...

NODE.JS STRUCTURE

```

├── server.js      → Serveur principal
├── socket_server.js → Serveur WebSocket pour temps réel
├── notifications.js → Système de notifications
├── email_service.js → Envoi d'emails
├── sms_service.js  → Envoi de SMS (pour alertes)
├── export_service.js → Service d'export avancé
├── background_jobs.js → Tâches en arrière-plan
└── api_gateway.js  → Gateway API pour microservices
  
```

...

Exemples concrets de ce que Node.js fait:

A. NOTIFICATIONS EN TEMPS RÉEL

```

````javascript

// socket_server.js - Communication temps réel

const io = require('socket.io')(3001);

io.on('connection', (socket) => {

 console.log('Nouveau client connecté'); // Écoute

 les événements RH socket.on('nouveau_employe',

 (employeeData) => {

 // Notifie tous les administrateurs io.emit('notification', { type:

 'nouveau_employe', message: `Nouvel employé: ${employeeData.nom}

 ${employeeData.prenom}`, data: employeeData

 });

 }); socket.on('paie_calculée', (paieData) => { //

 Notifie le service comptabilité

 io.to('comptabilite').emit('paie_prete', paieData);

 }); socket.on('alerte_conge', (congeData) => { // Alertes pour les congés à

 approuver io.to(`manager_${congeData.manager_id}`).emit('conge_a_approuver',

 congeData); });

 });

// Dans le frontend JavaScript const

socket = io('http://localhost:3001');

// Écoute les notifications

socket.on('notification', (data) => {

 showNotification(data.message); // Si c'est une

 alerte importante, faire clignoter l'onglet if

```

```

(data.type === 'urgence') { document.title = '
ALERTE - ' + document.title;

}

});
...

```

## B. SERVICE D'ENVOI D'EMAILS ET SMS

```

```javascript

// email_service.js - Envoi d'emails automatiques const
nodemailer = require('nodemailer');

class EmailService {  constructor() {

this.transporter = nodemailer.createTransport({
service: 'gmail',    auth: {    user:
'rh@entreprise.bj',    pass: 'motdepasse'
    }
});

}    async sendWelcomeEmail(employee) {
const mailOptions = {    from: 'RH Entreprise
<rh@entreprise.bj>',    to: employee.email,
subject: 'Bienvenue dans notre entreprise',    html:
,

```

```

    <h1>Bienvenue ${employee.prenom}!</h1>

```

```

    <p>Votre compte a été créé avec succès.</p>

```

```

    <p>Vos identifiants de connexion:</p>    <ul>

```

```

        <li>Email: ${employee.email}</li>

```

```

        <li>Mot de passe temporaire: ${employee.temp_password}</li>

```


<p>Veuillez changer votre mot de passe à la première connexion.</p>

```
    };

    await
this.transporter.sendMail(mailOptions);

    }    async sendPayslip(employee, payslip) {    // Envoie la fiche de paie
par email    const mailOptions = {    to: employee.email,    subject:
`Fiche de paie - ${payslip.mois}/${payslip.annee}`,    attachments: [{
filename: `fiche_paie_${payslip.mois}_${payslip.annee}.pdf`,    content:
payslip.pdf_buffer

    }]

    };

    await this.transporter.sendMail(mailOptions);

    }

}
```

// sms_service.js - Envoi d'SMS pour les alertes class

```
SMSService {    async
```

```
sendContractRenewalReminder(employee) {
```

```
    // Service SMS local au Bénin
```

```
    const message = `Bonjour ${employee.prenom}, votre contrat se termine le
${employee.fin_contrat}. Contactez le service RH.`;
```

```
    // Intégration avec un service SMS béninois
```

```
    await fetch('https://api.sms-benin.bj/send', {
```

```
    method: 'POST',    body: JSON.stringify({
```

```
    phone: employee.telephone,    message:
```

```
    message
```

```

    })

  });

}

}

...

```

C. TRAITEMENT PAR LOTS ET BACKGROUND JOBS

```

````javascript

// background_jobs.js - Tâches en arrière-plan

const cron = require('node-cron');

class BackgroundJobs {
 init() {

 // Tous les 1er du mois à minuit: calcul de la paie
 cron.schedule('0 0 1 * *', () => { this.calculateMonthlyPayroll();

 });

 // Tous les jours à 8h: rappels de réunion cron.schedule('0 8
 * * *', () => { this.sendMeetingReminders();

 });

 // Tous les lundis: rapports hebdomadaires cron.schedule('0
 9 * * 1', () => { this.generateWeeklyReports();

 });

 } async calculateMonthlyPayroll() {
 console.log('Début du calcul de la paie mensuelle...');
 }
}

```

```

 // Récupère tous les employés actifs const employees = await
fetchFromDjangoAPI('/api/employees/active/');

 for (const employee of employees) { // Calcule
la paie pour chaque employé const payslip = await
calculatePayroll(employee); // Génère le PDF
const pdf = await generatePayslipPDF(payslip);

 // Envoie par email await
emailService.sendPayslip(employee, {
 ...payslip, pdf_buffer:
pdf
 });

 // Met à jour la base de données Django await
updateDjangoPayroll(payslip);

 } console.log(`Paie calculée pour ${employees.length}
employés`);

 }

}

...

```

#### D. MICROSERVICE POUR LES RAPPORTS AVANCÉS

```

```javascript

// report_service.js - Génération de rapports complexes class ReportService {

async generateAnnualReport(year) {      // Récupère les données de Django

```

```

const data = await fetchFromDjangoAPI(`/api/reports/annual/${year}`); //
// Traitement complexe avec Node.js
const report = {
  summary: this.generateSummary(data),
  charts: this.generateCharts(data),
  statistics: this.calculateStatistics(data),
  recommendations: this.generateRecommendations(data)
};

// Génère le rapport en PDF
const pdf = await this.generatePDFReport(report);

// Envoie aux managers concernés
await this.distributeReport(report, pdf);

return report;
}

generateCharts(data) {
  // Utilise Chart.js ou autre bibliothèque
  return {
    payrollEvolution: this.createPayrollEvolutionChart(data),
    departmentDistribution: this.createDepartmentChart(data),
    turnoverRate: this.createTurnoverChart(data)
  };
}
}
...

```

Responsabilités de Node.js:

- Notifications en temps réel (WebSockets)
- Envoi d'emails automatiques

- Envoi de SMS (alertes importantes)
- Traitement par lots (paie mensuelle)
- Tâches planifiées (cron jobs)
- Microservices spécialisés
- Génération de rapports complexes
- Upload et traitement de fichiers
- API Gateway pour plusieurs services
- Monitoring et logs en temps réel

BASE DE DONNÉES

6. Base de données (SQLite/PostgreSQL)

Rôle principal: Stockage permanent de toutes les données

Ce qui sera stocké:

...

TABLES PRINCIPALES

└─ employees	→ Informations des employés
└─ contracts	→ Contrats de travail
└─ payroll	→ Historique des paies
└─ leave_requests	→ Demandes de congé
└─ attendances	→ Pointages et présences
└─ departments	→ Départements

└── positions → Postes et grades
└── evaluations → Évaluations de performance
└── users → Utilisateurs du système
...

Structure type:

```
```sql
```

```
-- Table employés (Bénin spécifique) CREATE TABLE employees (
 id SERIAL
 PRIMARY KEY, nom VARCHAR(100) NOT NULL, prenom VARCHAR(100) NOT
 NULL, cni_number VARCHAR(50) UNIQUE, -- Carte Nationale d'Identité
 cnss_number VARCHAR(50) UNIQUE, -- Numéro CNSS date_naissance
 DATE NOT NULL, lieu_naissance VARCHAR(200), telephone VARCHAR(20),
 email VARCHAR(100) UNIQUE, adresse TEXT, banque VARCHAR(100), rib
 VARCHAR(50), -- Relevé d'Identité Bancaire poste_id INTEGER
 REFERENCES positions(id), departement_id INTEGER REFERENCES
 departments(id), salaire_base DECIMAL(12, 2), -- En FCFA
 date_embauche DATE, date_fin_contrat DATE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Table paie avec calculs CNSS/IRPP CREATE TABLE payroll (
 id SERIAL
 PRIMARY KEY, employee_id INTEGER REFERENCES employees(id), mois
 INTEGER NOT NULL, annee INTEGER NOT NULL, salaire_brut DECIMAL(12,
 2), cnss_employe DECIMAL(12, 2), -- 4.5% cnss_employeur
 DECIMAL(12, 2), -- 6.5% irpp DECIMAL(12, 2), -- Calcul
 progressif primes DECIMAL(12, 2), deductions DECIMAL(12, 2),
 net_a_payer DECIMAL(12, 2), statut VARCHAR(20) DEFAULT 'en_attente', --
```

payé, en\_attente, annulé date\_paiement DATE, pdf\_path VARCHAR(255)

-- Chemin vers le PDF généré

);

...

---

COMMENT TOUT CELA TRAVAILLE ENSEMBLE

Scénario 1: Ajout d'un nouvel employé

...

1. UTILISATEUR: Remplit le formulaire HTML

↓

2. JAVASCRIPT: Valide les données en temps réel

↓

3. JAVASCRIPT: Envoie les données à Django via API

↓

4. DJANGO: Valide côté serveur, enregistre en base

↓

5. DJANGO: Déclenche des actions (génération contrat)

↓

6. NODE.JS: Envoie un email de bienvenue (asynchrone)

↓

7. NODE.JS: Envoie une notification en temps réel aux managers

↓

8. JAVASCRIPT: Met à jour l'interface sans rechargement

↓

9. BASE DE DONNÉES: Stocke toutes les informations

...

Scénario 2: Calcul de la paie mensuelle

...

1. CRON JOB (Node.js): Déclenché le 1er du mois



2. NODE.JS: Récupère tous les employés actifs via API Django



3. DJANGO: Fournit les données avec la logique métier



4. NODE.JS: Calcule la paie pour chaque employé



5. DJANGO: Enregistre les résultats en base de données



6. NODE.JS: Génère les PDF de fiche de paie



7. NODE.JS: Envoie les fiches par email aux employés



8. NODE.JS: Envoie des notifications de confirmation



9. JAVASCRIPT: Met à jour le dashboard en temps réel

...

Scénario 3: Demande de congé



...

1. EMPLOYÉ: Remplit le formulaire HTML de congé  
↓
2. JAVASCRIPT: Calcule les jours disponibles  
↓
3. DJANGO: Valide selon les règles métier  
↓
4. BASE DE DONNÉES: Enregistre la demande  
↓
5. NODE.JS: Envoie une notification WebSocket au manager ↓
6. MANAGER: Reçoit la notification en temps réel  
↓
7. MANAGER: Approuve/Rejette via l'interface  
↓
8. DJANGO: Met à jour le statut  
↓
9. NODE.JS: Envoie un SMS de confirmation à l'employé  
↓
10. JAVASCRIPT: Met à jour le calendrier des congés

...

---

## RÉPARTITION DES TÂCHES

Tâche Technologie Pourquoi

Structure des pages HTML Définition sémantique du contenu

Design et apparence CSS Style, responsive, thèmes

Interactivité immédiate JavaScript Réactivité côté client

Calculs complexes Python/Django Puissance mathématique, logique métier

Base de données Django Models ORM puissant, migrations

Administration Django Admin Interface prête à l'emploi

Temps réel Node.js WebSockets, événements instantanés

Emails/SMS Node.js Asynchrone, non-bloquant

Tâches planifiées Node.js Cron jobs, background processing

Génération PDF Python/Django Librairies matures (ReportLab)

API REST Django REST Framework robuste pour API

Sécurité Django Authentification, permissions, CSRF

---

## POURQUOI CETTE ARCHITECTURE POUR LE BÉNIN?

Avantages spécifiques:

1. Python/Django pour la logique métier:

- Calculs précis de paie selon lois béninoises
- Gestion des spécificités locales (CNSS, IRPP)
- Génération de documents légaux

2. Node.js pour la communication:

- SMS pour les employés sans email
- Notifications instantanées importantes
- Intégration avec services locaux

3. JavaScript/HTML/CSS pour l'interface:

- Léger, fonctionne sur connexions lentes
- Accessible depuis mobile (très utilisé au Bénin)
- Pas besoin de plugins spéciaux

#### 4. Base de données unique:

- Centralise toutes les données RH
- Backup facile
- Conformité avec protection des données

#### Adaptation au contexte béninois:

- Support offline partiel (localStorage)
- Interface légère pour bas débit
- SMS intégré pour notifications urgentes
- Conforme Code du travail béninois
- Multi-langue (français + langues locales)
- Support mobile-first

---

#### EN RÉSUMÉ

- HTML → La structure, les formulaires, le contenu
- CSS → Le design, l'expérience utilisateur, le responsive
- JavaScript → L'interactivité, les calculs côté client, la communication
- Python/Django → Le cerveau, la logique métier, l'administration, la sécurité
- Node.js → Les communications temps réel, les services externes, le traitement asynchrone •

Base de données → La mémoire, le stockage permanent

Chaque technologie a un rôle précis et complémentaire. Ensemble, elles forment une application RH complète, moderne et adaptée au contexte béninois.

## PHASE 1: PRÉPARATION (Jour 1)

### Étape 1.1: Organise ton espace de travail

Action concrète à faire:

#### 1. Sur ton Bureau Windows:

- Positionne ta souris sur le fond d'écran (zone vide)
- Fais un CLIC DROIT n'importe où sur le bureau
- Dans le menu qui apparaît, déplace ta souris vers "Nouveau"
- Dans le sous-menu, clique sur "Dossier"

#### 2. Nomme ce dossier:

- Le dossier apparaît avec le nom "Nouveau dossier" en surbrillance
- Tape directement: `rh_software`
- Appuie sur Entrée pour valider

Vérification:

- Tu dois voir sur ton bureau: `rh_software`

### Étape 1.2: Ouvre le dossier dans VS Code

Action concrète à faire:

1. Double-clique sur le dossier rh\_software que tu viens de créer
2. Dans l'Explorateur Windows (la fenêtre qui s'ouvre), clique en haut dans la barre d'adresse
3. Tape: cmd puis appuie sur Entrée
  - Une fenêtre noire (terminal) s'ouvre

Alternative plus simple:

1. Ouvre VS Code (icône bleue sur le bureau)
2. Va dans Fichier → Ouvrir le dossier (Ctrl+Maj+O)
3. Navigue jusqu'au Bureau et sélectionne rh\_software
4. Clique sur "Sélectionner le dossier"

Étape 1.3: Écris ton cahier des charges SIMPLE

Dans VS Code:

1. À gauche, dans l'explorateur, tu vois ton dossier rh\_software
2. Clique sur l'icône "Nouveau fichier" (carré avec + en haut à droite)
3. Nomme-le: cahier\_charges.txt
4. Appuie sur Entrée
5. Clique dans le fichier ouvert à droite et écris:

...

MON LOGICIEL RH DOIT AVOIR:

PAGE 1 - CONNEXION:

- Champ "nom d'utilisateur"
- Champ "mot de passe"

- Bouton "Se connecter"

#### PAGE 2 - TABLEAU DE BORD:

- Nombre total d'employés
- Derniers contrats ajoutés
- Alertes (anniversaires, fin de contrat)

#### PAGE 3 - EMPLOYÉS:

- Tableau avec: Nom, Prénom, Poste, Téléphone, Email
- Bouton "Ajouter un employé"
- Bouton "Voir fiche" à côté de chaque employé

#### PAGE 4 - CONTRATS:

- Type de contrat (CDI, CDD, Stage)
- Date de début, date de fin
- Salaire de base
- Bouton "Renouveler"

#### PAGE 5 - PAIE:

- Sélectionner un employé
- Sélectionner un mois
- Voir: Salaire brut, CNSS, IRPP, Net à payer
- Bouton "Générer fiche PDF"

...

Sauvegarde: Appuie sur Ctrl+S

#### PHASE 2: INSTALLATION DES OUTILS (Jour 2)

## Étape 2.1: Installe VS Code - DÉTAILLÉ

Action étape par étape:

1. Ouvre Google Chrome ou Firefox
  2. Dans la barre d'adresse, tape: `code.visualstudio.com`
  3. Page chargée: Tu vois un gros bouton bleu "Download for Windows"
  4. Clique dessus → Le téléchargement commence
  5. Quand c'est fini, en bas à gauche de Chrome, clique sur le fichier .exe
    - Ou va dans ton dossier Téléchargements et double-clique sur `VSCodeUserSetup-x64-*.exe`
- Installation:

6. Une fenêtre s'ouvre: "User Account Control"

- Clique sur "Oui"

1. Écran de bienvenue: Clique sur "Suivant"
2. Contrat de licence: Cocher "J'accepte" → "Suivant"
3. Choix du dossier d'installation: Ne change rien → "Suivant"
4. Sélection des tâches:
  - "Créer une icône sur le bureau" (IMPORTANT)
  - "Ajouter à PATH" (TRÈS IMPORTANT)
  - "Enregistrer tous les types de fichiers avec VS Code"
  - Clique "Suivant"
5. Clique "Installer"
6. À la fin, décoche "Lancer VS Code" et clique "Terminer"

## Étape 2.2: Installe Python - DÉTAILLÉ

Action étape par étape:

1. Ouvre ton navigateur à: [python.org](https://python.org)
2. Passe ta souris sur "Downloads" (en haut)
3. Dans le menu qui descend, clique sur "Windows"
4. Tu vois: "Python 3.x.x" avec un bouton jaune "Download Python 3.x.x"
  - Clique sur le bouton jaune
5. Le fichier s'installe: `python-3.x.x-amd64.exe`
6. Double-clique sur ce fichier dans tes téléchargements

Installation:

7. Fenêtre d'installation: Coche IMPORTANT:

- "Add python.exe to PATH" (en bas)
- Clique sur "Install Now"

1. Barre de progression → Attends que ça finisse
2. À la fin: "Setup was successful"
  - Clique sur "Close"

Étape 2.3: Vérifie que Python est installé - DÉTAILLÉ

Dans VS Code:

1. Ouvre VS Code (icône bleue sur le bureau)
2. Ouvre le terminal:
  - Va dans le menu Affichage → Terminal



- OU appuie sur Ctrl + ù (touche à gauche du 1)
- OU clique sur Terminal → Nouveau terminal

3. Dans le terminal noir: ````bash python --version`

...

Tape exactement `python --version` puis Entrée

Résultat attendu: Python 3.x.x

Si erreur: Essaie `python3 --version` ou `py --version`

4. Vérifie pip (gestionnaire de paquets):

````bash`

`pip --version`

...

Doit afficher `pip 23.x.x`

PHASE 3: DJANGO - PREMIERS PAS (Jour 3)

Étape 3.1: Ouvre ton projet dans VS Code

Action concrète:

1. Dans VS Code: Fichier → Ouvrir le dossier
2. Navigue vers: `C:\Users\TON_NOM\Desktop\rh_software`
3. Clique sur `rh_software`
4. Clique sur "Sélectionner le dossier"
5. À gauche, tu vois maintenant: EXPLORATEUR → RH_SOFTWARE

Étape 3.2: Crée ton environnement virtuel

Pourquoi? C'est comme une boîte isolée pour ton projet, pour éviter les conflits.

Dans le terminal VS Code:

1. Assure-toi d'être dans rh_software: ````bash cd`

`Desktop/rh_software`

`````

2. Crée l'environnement:

````bash`

`python -m venv venv`

`````

- venv = nom de ton environnement •

Ça crée un dossier venv dans ton projet

3. Active l'environnement (Windows): ````bash`

`venv\Scripts\activate`

`````

Si erreur "Execution Policy":

1. Ouvre le terminal en tant qu'administrateur:

- Menu Démarrer → tape "cmd" → clic droit → "Exécuter en tant qu'administrateur"

2. Tape:

````powershell`

`Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

`````

3. Répond Y puis Entrée

4. Retourne dans VS Code et réessaye

4. Vérifie que (venv) apparaît au début de la ligne

Étape 3.3: Installe Django

Dans le terminal (avec (venv) visible):

```
```bash pip install  
django
```
```

Détails de ce qui se passe:

- pip = outil d'installation Python
- install = commande pour installer
- django = nom du framework

Vérification:

```
```bash django-admin --  
version
```
```

Doit afficher 4.x.x

PHASE 4: CRÉE TON PROJET (Jour 4)

Étape 4.1: Crée le projet Django

Dans le terminal:

```
```bash django-admin startproject
```

```
rh_benin
```

```
```
```

Ce que cette commande fait:

- Crée un dossier rh_benin avec toute la structure de base
- startproject = commande Django pour créer un nouveau projet
- rh_benin = nom de ton projet

Vérifie la structure créée:

```
```bash dir
```

```
rh_benin
```

```
```
```

Tu dois voir:

- manage.py (fichier principal)
- rh_benin/ (sous-dossier)

Étape 4.2: Entre dans le dossier

```
```bash cd
```

```
rh_benin
```

```
```
```

Étape 4.3: Lance le serveur pour tester

```
'''bash python manage.py
```

```
runserver
```

```
'''
```

Ce qui se passe:

- Django démarre un serveur web sur ton PC • Il écoute sur le port 8000
- C'est un serveur de développement (pour tester)

Résultat dans le terminal:

```
'''
```

```
System check identified no issues (0 silenced).
```

```
Django version 4.x.x, using settings 'rh_benin.settings'
```

```
Starting development server at http://127.0.0.1:8000/
```

```
Quit the server with CTRL-BREAK.
```

```
'''
```

Étape 4.4: Vois ton site

1. Ouvre ton navigateur (Chrome/Firefox)
2. Dans la barre d'adresse, tape: `http://127.0.0.1:8000`
3. Tu dois voir: Une fusée avec "The install worked successfully!"
4. Félicitations! Ton premier site Django tourne

Étape 4.5: Arrête le serveur

Dans le terminal VS Code:

- Appuie sur Ctrl + C
- Le serveur s'arrête, tu reviens à la ligne de commande

PHASE 5: CRÉE L'APPLICATION EMPLOYÉS (Jour 5)

Étape 5.1: Qu'est-ce qu'une "app" Django?

C'est un module indépendant. Ton projet peut avoir plusieurs apps:

- employees = gestion des employés
- payroll = gestion de la paie
- contracts = gestion des contrats

Étape 5.2: Crée l'app "employees"

Dans le terminal (toujours dans rh_benin):

```
```bash python manage.py startapp  
employees
```
```

Ce qui est créé:

```
...
```

employees/

|—— migrations/ # Pour les changements de base de données

|—— __init__.py # Fichier vide qui dit que c'est un module Python

```

└── admin.py      # Configuration de l'interface admin
└── apps.py       # Configuration de l'app
└── models.py     **ICI ON VA CRÉER NOS TABLES**
└── tests.py      # Pour les tests (plus tard)
└── views.py      **ICI ON VA CRÉER NOS PAGES WEB**
...

```

Étape 5.3: Dis à Django d'utiliser cette app

1. Dans VS Code, à gauche, ouvre: rh_benin/settings.py
2. Cherche la section INSTALLED_APPS (vers la ligne 33)
3. Ajoute 'employees', à la fin de la liste:

```

```python
INSTALLED_APPS = [
 'django.contrib.admin',
 'django.contrib.auth',
 'django.contrib.contenttypes',
 'django.contrib.sessions',
 'django.contrib.messages',
 'django.contrib.staticfiles',
 'employees', # <-- AJOUTE CETTE LIGNE ICI
]
...

```

IMPORTANT: La virgule à la fin est importante!

PHASE 6: CRÉE TA PREMIÈRE TABLE (Jour 6)

## Étape 6.1: Comprends les modèles Django

Un modèle = une table dans la base de données

- Employee = table "employees"
- Chaque attribut = une colonne dans la table

## Étape 6.2: Ouvre et modifie employees/models.py

1. Dans VS Code, ouvre: employees/models.py
2. Efface tout le contenu
3. Copie-colle ce code:

```
```python from django.db
```

```
import models
```

```
class Employee(models.Model):  # Informations personnelles  first_name =
models.CharField(max_length=100, verbose_name="Prénom")  last_name =
models.CharField(max_length=100, verbose_name="Nom")

    # Contact  email = models.EmailField(verbose_name="Email", unique=True)
phone = models.CharField(max_length=20, verbose_name="Téléphone", blank=True)

    # Professionnel  position = models.CharField(max_length=100,
verbose_name="Poste")

    department  =      models.CharField(max_length=100,      verbose_name="D é
partement", default="Non spécifié")

    # Salaire  salary =
models.DecimalField(
```



```

max_digits=10,

decimal_places=2,

verbose_name="Salaire de base
(FCFA)"

)

# Dates importantes  hire_date = models.DateField(verbose_name="Date d'embauche")

birth_date = models.DateField(verbose_name="Date de naissance", null=True, blank=True)

# Statut  is_active = models.BooleanField(default=True, verbose_name="Actif")


# Photo (optionnel)

photo = models.ImageField(upload_to='employees/', blank=True, null=True,
verbose_name="Photo")


# Métadonnées  created_at =
models.DateTimeField(auto_now_add=True)  updated_at =
models.DateTimeField(auto_now=True)

def __str__(self):

return f"{self.first_name} {self.last_name} - {self.position}"

def full_name(self):

return f"{self.last_name.upper()} {self.first_name}"

class Meta:

verbose_name = "Employé"  verbose_name_plural

= "Employés"  ordering = ['last_name', 'first_name']

...

```

Étape 6.3: Explication des champs

- CharField = texte court (nom, prénom)
- EmailField = email avec validation automatique
- DecimalField = nombre avec décimales (pour l'argent)
- DateField = date (sans heure)
- BooleanField = vrai/faux (case à cocher)
- ImageField = pour uploader des photos
- auto_now_add=True = date de création automatique
- auto_now=True = date de modification automatique

Étape 6.4: Installe Pillow pour les images

```
```bash pip
install Pillow
```
```

Étape 6.5: Crée les migrations

Migrations = instructions pour créer/modifier la base de données

```
```bash python manage.py makemigrations
employees
```
```

Résultat attendu:

```
```
```

Migrations for 'employees':

employees/migrations/0001\_initial.py

- Create model Employee

'''

Étape 6.6: Applique les migrations

```
'''bash python manage.py
```

```
migrate
```

```
'''
```

Résultat:

'''

Operations to perform:

Apply all migrations: admin, auth, contenttypes, employees, sessions

Running migrations:

Applying employees.0001\_initial... OK

'''

FÉLICITATIONS! Ta table "Employee" est créée dans la base de données!

PHASE 7: INTERFACE ADMIN (Jour 7)

Étape 7.1: Active l'interface admin pour Employee

Ouvre employees/admin.py et remplace par:

```
'''python from django.contrib
```

```
import admin from .models
```

```
import Employee
```

```
class EmployeeAdmin(admin.ModelAdmin):

 # Ce qui s'affiche dans la liste list_display = ('full_name', 'position', 'department',
'salary', 'hire_date', 'is_active')

 # Filtres sur la droite list_filter = ('department',
'position', 'is_active')

 # Barre de recherche search_fields = ('first_name',
'last_name', 'email', 'phone')

 # Organisation du formulaire d'édition fieldsets
= (
 ('Informations personnelles', {
 'fields': ('first_name', 'last_name', 'birth_date', 'photo')
 }),
 ('Informations professionnelles', {
 'fields': ('position', 'department', 'salary', 'hire_date')
 }),
 ('Contact', {
 'fields': ('email', 'phone')
 }),
 ('Statut', {
 'fields': ('is_active',)
 }),
)
```

```
Champs en lecture seule readonly_fields =
('created_at', 'updated_at')
```

```
Enregistre le modèle avec sa configuration
admin.site.register(Employee, EmployeeAdmin)
...
```

Étape 7.2: Crée un super-utilisateur

```
```bash python manage.py  
createsuperuser  
...
```

Réponds aux questions:

```
...
```

Nom d'utilisateur: admin

Adresse courriel: admin@entreprise.bj

Password: admin123 (tu ne vois rien quand tu tapes, c'est normal)

Password (again): admin123

```
...
```

Appuie sur Entrée après chaque ligne

Étape 7.3: Lance le serveur et connecte-toi

```
```bash python manage.py  
runserver
```

...

Dans ton navigateur:

1. Va à: `http://127.0.0.1:8000/admin`

2. Connecte-toi avec:

- Utilisateur: admin
- Mot de passe: admin123

Étape 7.4: Explore l'admin

1. Tu vois: "Administration de Django"

2. Clique sur "Employés" dans la section EMPLOYEES

3. Clique sur "AJOUTER EMPLOYÉ" en haut à droite

4. Remplis le formulaire:

- Prénom: Koffi
- Nom: Doe
- Email: koffi@entreprise.bj
- Poste: Développeur
- Salaire: 300000
- Date d'embauche: 2024-01-15

5. Clique sur "ENREGISTRER"

6. Ajoute 2-3 employés pour tester

Étape 7.5: Configure la langue française

Dans `rh_benin/settings.py`:

```
```python

# Tout en haut du fichier import os from

django.utils.translation import gettext_lazy as _


# Dans la section LANGUAGE_CODE

LANGUAGE_CODE = 'fr-fr'


# Dans la section TIME_ZONE (pour le Bénin)

TIME_ZONE = 'Africa/Porto-Novo'


# Dans INSTALLED_APPS, ajoute 'modeltranslation' si tu veux la traduction
...

```

Redémarre le serveur (Ctrl+C puis python manage.py runserver)

PHASE 8: TA PREMIÈRE PAGE WEB (Jour 8)

Étape 8.1: Crée une vue simple

Ouvre employees/views.py et remplace par:

```
```python

from django.shortcuts import render,
get_object_or_404 from django.http import
HttpResponse from .models import Employee

def employee_list(request):

```

```

"""

Affiche la liste de tous les employés """

Récupère tous les employés actifs employees =
Employee.objects.filter(is_active=True).order_by('last_name')

Compte le nombre d'employés total_employees
= employees.count()

Prépare les données à envoyer au template context
= {
 'employees': employees,
 'total_employees': total_employees,
 'page_title': 'Liste des Employés',
}

Renvoie la page HTML avec les données return
render(request, 'employees/list.html', context)

def employee_detail(request, employee_id):
 """

 Affiche les détails d'un employé spécifique

 """

 # Récupère l'employé par son ID, ou erreur 404 si non trouvé employee
 = get_object_or_404(Employee, id=employee_id)

 context = {
 'employee': employee,

```



```

 'page_title': f'Fiche de {employee.full_name()}',
 }
 return render(request, 'employees/detail.html',
context)

def home(request):
 """
 Page d'accueil du module RH
 """
 total_employees =
Employee.objects.filter(is_active=True).count()

 context = {
 'total_employees': total_employees,
 'welcome_message': 'Bienvenue dans le Système de Gestion RH',
 'page_title': 'Accueil - Gestion RH Bénin',
 }
 return render(request, 'employees/home.html',
context)

```

## Étape 8.2: Crée le système de templates

Structure des dossiers:

1. Dans le dossier employees, crée un dossier templates
2. Dans templates, crée un dossier employees
3. Dans employees/templates/employees, crée ces fichiers:
  - base.html (template de base)
  - home.html (page d'accueil)
  - list.html (liste des employés)

- detail.html (détail d'un employé)

### Étape 8.3: Crée le template de base (base.html)

```

`html
<!DOCTYPE html>
<html lang="fr">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>{% block title %}Gestion RH Bénin{% endblock %}</title>

 <!-- CSS de base -->
 <style>
 * {
 margin: 0;
 padding: 0;
 box-sizing:
border-box;
 font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-
serif;
 }
 body {
 background-color: #f5f7fa;
 color:
#333;
 line-height: 1.6;
 }

 /* Header */
 .header {
 background: linear-
gradient(135deg, #2c3e50, #3498db);
 color: white;
 padding: 1rem 2rem;
 box-shadow: 0 2px 10px rgba(0,0,0,0.1);
 }

 .header h1 {
 font-
size: 1.8rem;
 display: flex;
 align-items: center;
 gap:
10px;
 }

```

```

.header h1::before {
content: " "; font-size: 1.5rem;

}

/* Navigation */ .navbar { background-
color: white; padding: 0.8rem 2rem;
border-bottom: 1px solid #e1e5e9; display: flex;
gap: 2rem; box-shadow: 0 1px 3px
rgba(0,0,0,0.05);

}

.navbar a { color:
#2c3e50; text-decoration:
none; padding: 0.5rem 1rem;
border-radius: 4px; transition:
all 0.3s; display: flex;
align-items: center; gap: 8px;

}

.navbar a:hover {

background-color: #3498db; color:
white;

}

.navbar a.active { background-
color: #2c3e50; color: white;

}

```

```

/* Contenu principal */

.container {
 max-width:
1200px;
 margin: 2rem auto;
padding: 0 1rem;
}

.content {
 background-color: white;
border-radius: 8px;
padding: 2rem;
box-shadow: 0 3px 15px rgba(0,0,0,0.08);
}

/* Cartes */
.card {
 background: white;
border-radius: 8px;
padding: 1.5rem;
margin-bottom: 1.5rem;
box-shadow: 0 2px 8px
rgba(0,0,0,0.05);
border-left: 4px solid
#3498db;
}

/* Boutons */

.btn {
 display: inline-flex;
align-items: center;
gap: 8px;
padding:
0.6rem 1.2rem;
background-color:
#3498db;
color: white;
text-decoration: none;
border-radius: 4px;
border: none;
cursor: pointer;
font-size: 0.95rem;
transition:
background-color 0.3s;
}

```

```

 .btn:hover { background-
color: #2980b9;

 }

 .btn-success { background-
color: #27ae60;

 }

 .btn-success:hover {
background-color: #219653;

 }

/* Tableaux */

table { width: 100%;
border-collapse: collapse;
margin-top: 1rem;

 } th { background-color:
#f8f9fa; padding: 1rem; text-align:
left; font-weight: 600; color:
#2c3e50; border-bottom: 2px solid
#e1e5e9;

 } td { padding: 1rem;
border-bottom: 1px solid #e1e5e9;

 } tr:hover {
background-color: #f8f9fa;

 }

/* Footer */ .footer {
text-align: center;

padding: 2rem; color:

```

```

#7f8c8d; font-size:
0.9rem; border-top: 1px
solid #e1e5e9; margin-
top: 3rem;
 }

 /* Messages */ .message {
padding: 1rem; border-radius:
4px; margin-bottom: 1rem;
 }

 .message.success {
background-color: #d4edda; color:
#155724; border: 1px solid #c3e6cb;
 }

 /* Responsive */

 @media (max-width: 768px) {
.container { padding: 0.5rem;
 }

 .content {
padding: 1rem;
 }

 table {
display: block; overflow-x:
auto;
 }
 }

</style>

```

```

<!-- Bloc pour le CSS additionnel -->

{% block extra_css %}{% endblock %}

</head>

<body>

 <!-- Header -->

 <header class="header">

 <h1> Gestion RH - Entreprise Bénin</h1>

 </header>

 <!-- Navigation -->

 <nav class="navbar">

 Accueil

 Employés

 Administration

 {{ user.username|default:"Invité" }}

 </nav>

 <!-- Contenu principal -->

```

```
<div class="container">

 <!-- Titre de page -->

 <h2 style="color: #2c3e50; margin-bottom: 1.5rem;">

 {% block page_title %}{% endblock %}

 </h2>

 <!-- Messages -->

 {% if messages %}

 {% for message in messages %}

 <div class="message {{ message.tags }}">

 {{ message }}

 </div>

 {% endfor %}

 {% endif %}

 <!-- Contenu spécifique à chaque page -->

 <div class="content">

 {% block content %}

 <!-- Le contenu des pages enfants va ici -->

 {% endblock %}

 </div>

</div>

<!-- Footer -->

<footer class="footer">

 <p>Système de Gestion RH - Développé au Bénin © 2024</p>

 <p>Conforme à la législation du travail béninois</p>

</footer>
```



```

<!-- JavaScript -->

{% block extra_js %}{% endblock %}

</body>

</html>

...

```

Étape 8.4: Crée la page d'accueil (home.html)

```

```html

{% extends 'employees/base.html' %}

{% block title %}Accueil - Gestion RH{% endblock %}

{% block page_title %}Tableau de Bord RH{% endblock %}

{% block content %}

<div class="card">

    <h3 style="color: #2c3e50; margin-bottom: 1rem;"> Statistiques</h3>

    <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 1.5rem; margin-top: 1.5rem;">

        <div style="text-align: center; padding: 1.5rem; background: #e8f4fc; border-radius: 8px;">

            <div style="font-size: 2.5rem; color: #3498db;"> </div>

            <h3 style="margin: 0.5rem 0; color: #2c3e50;">{{ total_employees }}</h3>

            <p style="color: #7f8c8d; margin: 0;">Employés actifs</p>

        </div>

        <div style="text-align: center; padding: 1.5rem; background: #e8f6f3; border-radius: 8px;">

```

```
<div style="font-size: 2.5rem; color: #27ae60;"> </div>
```

```
<h3 style="margin: 0.5rem 0; color: #2c3e50;">En développement</h3>
```

```
<p style="color: #7f8c8d; margin: 0;">Module Paie</p>
```

```
</div>
```

```
<div style="text-align: center; padding: 1.5rem; background: #fef9e7; border-radius: 8px;">
```

```
<div style="font-size: 2.5rem; color: #f39c12;"> </div>
```

```
<h3 style="margin: 0.5rem 0; color: #2c3e50;">Bientôt</h3>
```

```
<p style="color: #7f8c8d; margin: 0;">Gestion des contrats</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="card">
```

```
<h3 style="color: #2c3e50; margin-bottom: 1rem;"> Actions rapides</h3>
```

```
<div style="display: flex; gap: 1rem; flex-wrap: wrap; margin-top: 1rem;">
```

```
<a href="{% url 'employee_list' %}" class="btn">
```

```
    Voir tous les employés
```

```
</a>
```

```
<a href="/admin/employees/employee/add/" class="btn btn-success" target="_blank">
```

```
    Ajouter un employé
```

```
</a>
```

```
<a href="#" class="btn">
```

```
    Générer un rapport
```

```
</a>
```

```
</div>
```

```
</div>
```

```

<div class="card">

  <h3 style="color: #2c3e50; margin-bottom: 1rem;"> Guide de démarrage</h3>

  <ol style="margin-left: 1.5rem; line-height: 2;">

    <li>Ajoute tes employés via l'interface d'administration</li>

    <li>Consulte la liste des employés dans "Employés"</li>

    <li>Configure les paramètres de paie (à venir)</li>

    <li>Génère les fiches de paie (à venir)</li>

  </ol>

</div>

{% endblock %}

'''

```

Étape 8.5: Crée la page liste (list.html)

```

'''html

{% extends 'employees/base.html' %}


{% block title %}Liste des Employés{% endblock %}

{% block page_title %}Liste des Employés ({{ total_employees }}){% endblock %}


{% block content %}

<div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1.5rem;">

  <div>

    <a href="/admin/employees/employee/add/" class="btn btn-success" target="_blank">

      Ajouter un employé

    </a>

  </div>

</div>

```

</div>

<div>

<input type="text" id="searchInput" placeholder=" Rechercher un employé..."

style="padding: 0.6rem; border: 1px solid #ddd; border-radius: 4px; width: 250px;">

</div>

</div>

{% if employees %}

<table id="employeesTable">

<thead>

<tr>

<th>Nom & Prénom</th>

<th>Poste</th>

<th>Département</th>

<th>Salaire (FCFA)</th>

<th>Date d'embauche</th>

<th>Actions</th>

</tr>

</thead>

<tbody>

{% for employee in employees %}

<tr>

<td>

{{ employee.last_name|upper }}
{{ employee.first_name }}

</td>

<td>{{ employee.position }}</td>

<td>{{ employee.department }}</td>

```

<td style="font-weight: 600; color: #27ae60;">

    {{ employee.salary|floatformat:0 }}

</td>

<td>{{ employee.hire_date|date:"d/m/Y" }}</td>

<td>

    <a href="{% url 'employee_detail' employee.id %}"
class="btn" style="padding: 0.4rem 0.8rem; font-size: 0.9rem;">

        Voir

    </a>

    <a href="/admin/employees/employee/{{ employee.id }}/change/"
class="btn" style="padding: 0.4rem 0.8rem; font-size: 0.9rem; background: #f39c12;">

        Modifier

    </a>

</td>

</tr>

{% endfor %}

</tbody>

</table>

{% else %}

<div class="card" style="text-align: center; padding: 3rem;">

    <div style="font-size: 3rem; color: #bdc3c7;"></div>

    <h3 style="color: #7f8c8d; margin: 1rem 0;">Aucun employé trouvé</h3>

    <p>Commence par ajouter des employés via l'interface d'administration.</p>

    <a href="/admin/employees/employee/add/" class="btn btn-success"
style="margin-top: 1rem;" target="_blank">

        Ajouter le premier employé

    </a>

</div>

{% endif %}

```

```

<script>

// Fonction de recherche simple

document.getElementById('searchInput').addEventListener('keyup', function()

{   const searchText = this.value.toLowerCase();   const rows =

document.querySelectorAll('#employeesTable tbody tr');

    rows.forEach(row => {   const text =

row.textContent.toLowerCase();   row.style.display =

text.includes(searchText) ? '' : 'none';

    });

});

</script>

```

```

<style>

#employeesTable th:nth-child(1) { width: 25%; }

#employeesTable th:nth-child(2) { width: 20%; }

#employeesTable th:nth-child(3) { width: 15%; }

#employeesTable th:nth-child(4) { width: 15%; }

#employeesTable th:nth-child(5) { width: 15%; }

#employeesTable th:nth-child(6) { width: 10%; }

</style>

```

```
{% endblock %}
```

```
...
```

Étape 8.6: Crée la page détail (detail.html)

```

```html

{% extends 'employees/base.html' %}

```

```
{% block title %}{{ employee.full_name }} - Fiche Employé{% endblock %}
```

```
{% block page_title %}Fiche Employé: {{ employee.full_name }}{% endblock %}
```

```
{% block content %}
```

```
<div style="display: grid; grid-template-columns: 1fr 2fr; gap: 2rem;">
```

```
<!-- Colonne gauche: Informations générales -->
```

```
<div>
```

```
<div class="card">
```

```
<div style="text-align: center; margin-bottom: 1.5rem;">
```

```
{% if employee.photo %}
```

```

```

```
{% else %}
```

```
<div style="width: 150px; height: 150px; border-radius: 50%; background: #3498db; color: white; display: flex; align-items: center; justify-content: center; font-size: 3rem; margin: 0 auto;">
```

```
{{ employee.first_name|first|upper }} {{ employee.last_name|first|upper }}
```

```
</div>
```

```
{% endif %}
```

```
</div>
```

```
<h3 style="color: #2c3e50; text-align: center; margin-bottom: 1rem;">
```

```
{{ employee.first_name }} {{ employee.last_name|upper }}
```

```
</h3>
```

```
<div style="background: #f8f9fa; padding: 1rem; border-radius: 6px;">
```

```
<div style="display: flex; justify-content: space-between; margin-bottom: 0.5rem;">
```

```
Poste:
```

```
{{ employee.position }}
```

```
</div>
```

```
<div style="display: flex; justify-content: space-between; margin-bottom: 0.5rem;">
```

```
Département:
```

```
{{ employee.department }}
```

```
</div>
```

```
<div style="display: flex; justify-content: space-between; margin-bottom: 0.5rem;">
```

```
Statut:
```

```

```

```
{% if employee.is_active %}Actif{% else %}Inactif{% endif %}
```

```

```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="card">
```

```
<h4 style="color: #2c3e50; margin-bottom: 1rem;"> Contact</h4>
```

```
<p> Email: {{ employee.email }}</p>
```

```
<p> Téléphone: {{ employee.phone|default:"Non renseigné " }}</p>
```

```
</div>
```

```
</div>
```

```
<!-- Colonne droite: Détails financiers -->
```



```
<div>
```

```
<div class="card">
```

```
<h4 style="color: #2c3e50; margin-bottom: 1rem;"> Informations financiè res</h4>
```

```
<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">
```

```
<div style="background: #e8f6f3; padding: 1rem; border-radius: 6px;">
```

```
<div style="color: #7f8c8d; font-size: 0.9rem;">Salaire de base</div>
```

```
<div style="font-size: 1.5rem; font-weight: 700; color: #27ae60;">
```

```
{{ employee.salary|floatformat:0 }} FCFA
```

```
</div>
```

```
</div>
```

```
<div style="background: #e8f4fc; padding: 1rem; border-radius: 6px;">
```

```
<div style="color: #7f8c8d; font-size: 0.9rem;">CNSS Employ é
(4.5%)</div>
```

```
<div style="font-size: 1.5rem; font-weight: 700; color: #3498db;">
```

```
{{ employee.salary|floatformat:0|add:"0"|floatformat:"-4"|slice:"":-4 }} FCFA
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="card">
```

```
<h4 style="color: #2c3e50; margin-bottom: 1rem;"> Informations de carriè re</h4>
```

```
<div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">
```

```
<div>
```

```
<div style="color: #7f8c8d; font-size: 0.9rem;">Date d'embauche</div>
```

```
<div style="font-size: 1.1rem; font-weight: 600;">
```

```

 {{ employee.hire_date|date:"d/m/Y" }}

 </div>

</div>

<div>

 <div style="color: #7f8c8d; font-size: 0.9rem;">Date de naissance</div>

 <div style="font-size: 1.1rem; font-weight: 600;">

 {% if employee.birth_date %}

 {{ employee.birth_date|date:"d/m/Y" }}

 {% else %}

 Non renseigné

 {% endif %}

 </div>

</div>

</div>

</div>

<div class="card">

 <h4 style="color: #2c3e50; margin-bottom: 1rem;"> Actions</h4>

 <div style="display: flex; gap: 1rem; flex-wrap: wrap;">

 <a href="/admin/employees/employee/{{ employee.id }}/change/"
class="btn">

 ➡ Modifier la fiche

 Calculer la paie


```

Générer contrat

</a>

<a href="{% url 'employee\_list' %}" class="btn" style="background: #95a5a6;">

↩ Retour à la liste

</a>

</div>

</div>

</div>

</div>

{% endblock %}

...

## Étape 8.7: Configure les URLs

Crée un fichier employees/urls.py:

```
python from django.urls
```

```
import path from . import
```

```
views app_name =
```

```
'employees'
```

```
urlpatterns = [path('', views.home, name='home'), path('employees/',
```

```
views.employee_list, name='employee_list'), path('employees/<int:employee_id>/',
```

```
views.employee_detail, name='employee_detail'),
```

```
]
```

```
...
```

Modifie rh\_benin/urls.py:

```
```python from django.contrib import
admin from django.urls import path,
include from django.conf import
settings from django.conf.urls.static
import static

urlpatterns = [ path('admin/', admin.site.urls), path("",
include('employees.urls')), # Inclut les URLs de l'app employees
]

# Pour servir les fichiers médias en développement if
settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
```
```

Étape 8.8: Configure les fichiers statiques et médias

Dans rh\_benin/settings.py, ajoute à la fin:

```
```python
# Fichiers statiques (CSS, JS, images)
STATIC_URL = 'static/' STATICFILES_DIRS = [
os.path.join(BASE_DIR, 'employees/static'),
]
```

```
# Fichiers uploadés par les utilisateurs

MEDIA_URL = 'media/'

MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

```
# Crée le dossier media si il n'existe pas if

not os.path.exists(MEDIA_ROOT):

os.makedirs(MEDIA_ROOT)

'''
```

Étape 8.9: Teste tout!

1. Redémarre le serveur: ```bash python manage.py runserver`

'''

2. Ouvre 3 onglets dans ton navigateur:

- <http://127.0.0.1:8000/> → Page d'accueil
- <http://127.0.0.1:8000/employees/> → Liste des employés
- <http://127.0.0.1:8000/employees/1/> → Détail du premier employé
- <http://127.0.0.1:8000/admin/> → Interface admin

RÉSUMÉ DE CE QUE TU AS MAINTENANT

Serveur Django fonctionnel

Base de données avec table Employee

Interface admin complète en français

3 pages web fonctionnelles:

- Page d'accueil avec statistiques
- Liste des employés avec recherche

- Page détail employé avec photo

Design professionnel avec CSS

Navigation entre les pages

Système prêt pour extensions

PROCHAINE ÉTAPE IMMÉDIATE

Ajoute des données de test:

1. Connecte-toi à l'admin (/admin)
2. Ajoute 5-6 employés avec différentes informations
3. Teste la recherche dans la liste
4. Clique sur "Voir" pour chaque employé

Prochain module à créer: payroll (paie) avec calculs CNSS Bénin!

Si tu as des erreurs:

1. Copie le message d'erreur exact
2. Vérifie que tu as bien suivi chaque étape
3. Redémarre le serveur après chaque modification

FÉLICITATIONS! Tu as créé la base de ton système RH!